Raian Pial

Simulation of Heterogeneous Computing and Sensing

What problem are we investigating?

The problem I am investigating is how different computing systems handle data collected from sensors and how the location of data processing affects overall performance. In modern computing environments, sensors are used to collect information such as temperature, movement, vibration, and environmental conditions. These sensors often send information to computers that process the data and produce results. However, when multiple sensors are operating at the same time, the system may experience delays because of limited processing power, network communication, or a large amount of incoming data.

This problem relates to heterogeneous computing, which involves using different types of computing resources within the same system. For example, a network might include smaller computers located near sensors and more powerful servers located remotely. One approach is edge computing, where information is processed close to where it is collected. Another is cloud computing, where sensor data is transmitted to remote servers for processing. Each approach has advantages, but their performance depends on the conditions of the system. Processing information closer to its source can help reduce communication delays and network usage, although local computing resources may have their own limitations (Shi et al., 2016).

A concrete example would be a factory that uses vibration sensors to monitor equipment. Under normal conditions, the sensors continuously collect information and send it to a processing system. The system analyzes the readings and identifies possible problems with the equipment. If the information is processed quickly, the factory can receive timely notifications about unusual activity. However, if the number of sensors increases, the local computer might not have enough processing power to handle all the incoming readings. This could cause information to accumulate in a waiting queue, delaying the results. Alternatively, sending everything to a cloud server could reduce the processing workload on the local computer but introduce delays from network communication.

For example, suppose a local computer can process five sensor readings per second. If the sensors produce only two readings per second, the computer may handle them without significant waiting. However, if the sensors begin producing seven readings per second, the computer receives information faster than it can process it. The waiting queue would continue growing if this workload remained constant. A cloud server with greater processing capacity might handle the additional readings, but its response time would also depend on how long it

takes to transmit the information. These numbers are hypothetical and are used only to illustrate the problem, not to represent actual experimental results.

My project will investigate this tradeoff by simulating sensor data being processed locally or remotely. The main focus will be processing time, network delay, and the number of sensor tasks completed within a reasonable time. The project will not attempt to recreate an entire industrial monitoring system or build physical sensor hardware. Instead, it will use a simplified software simulation to understand which processing approach performs better under different conditions.

Who or what is affected by it?

This problem affects organizations and systems that depend on sensors to collect and process information. Examples include factories, smart buildings, transportation networks, and environmental monitoring systems. These applications may depend on timely information to monitor conditions or identify changes. When processing is delayed, the information provided to users may no longer represent the most recent conditions.

For example, in the factory scenario, delayed processing could prevent monitoring software from identifying unusual equipment behavior promptly. If sensor readings are waiting in a queue, the system may be analyzing older information while newer readings continue arriving. Employees relying on the monitoring system could therefore receive delayed notifications or make decisions using outdated information. This does not necessarily mean equipment will fail, but it creates a risk that important changes will not be recognized quickly enough.

The computing infrastructure is also affected. A local computer with limited processing capacity may become overloaded when the number of sensor tasks increases. A remote server may have greater processing power, but transmitting information requires network resources. These challenges are especially relevant to Internet of Things (IoT) systems, which consist of physical devices that collect and exchange information through networks. Research on IoT resource management has identified communication delays, network congestion, and computing limitations as important factors affecting system performance (Gupta et al., 2017).

Understanding these effects is important because organizations need to decide where their data should be processed. Choosing a processing approach without considering the workload and network conditions could result in unnecessary delays or inefficient resource usage. My project will help explain these differences through controlled simulations rather than assuming that one computing approach is always better.

What evidence do you currently have that it is a real problem?

The challenges associated with processing sensor data have been studied in distributed computing research, particularly as more devices have become connected through networks. As

the number of sensors increases, computing systems must manage larger amounts of incoming data while maintaining reasonable response times. Processing everything through remote servers can create additional communication overhead, while relying entirely on local devices may introduce limitations related to available computing resources. These concerns make the location of data processing an important factor when designing sensor-based systems.

Edge computing has gained attention partly because sending data to distant servers is not always the most efficient approach. Shi et al. (2016) discuss how processing information closer to where it is generated can reduce network communication and support applications that require timely responses. However, this does not mean edge computing will always perform better. Its effectiveness depends on factors such as workload, computing capacity, and network conditions. In situations where local devices cannot keep up with incoming data, processing delays may still occur even when network communication is reduced.

The challenges of assigning computing resources to sensor applications have also encouraged the development of simulation tools. Gupta et al. (2017) developed iFogSim, a framework that allows researchers to model and evaluate computing environments involving IoT devices, edge devices, and cloud servers. The framework supports investigating factors such as processing latency, network congestion, energy consumption, and resource allocation. This demonstrates why simulation is useful for examining distributed computing systems before investing in physical hardware or implementing a complete network.

Existing technologies also show that processing sensor information locally is a practical approach. Amazon Web Services (AWS) provides AWS IoT Greengrass, which allows connected devices to process data locally while maintaining communication with cloud services. This type of system can reduce unnecessary data transmission and allow some operations to continue without constant communication with a remote server. Its existence demonstrates that organizations have practical reasons to use local computing resources alongside cloud infrastructure.

Although these findings establish that the general problem is worth investigating, they do not determine how my particular simulation will behave. I have not yet built the simulation, measured processing delays, or tested different sensor workloads. At this stage, the expectation that higher workloads will increase local waiting times is based on the relationship between task arrival rates and processing capacity. Similarly, I expect that increasing network delay will affect the response time of cloud processing. Both expectations still need to be tested within the proposed model.

The main evidence still missing is experimental data showing how these factors interact under controlled conditions. By developing a simulation and measuring its behavior, I will be able to move beyond general assumptions and determine which processing approach performs better under the conditions being tested.

What solutions already exist?

Two common approaches for processing sensor information are centralized cloud computing and edge computing. Both are used in distributed systems, but they handle data differently and involve different performance tradeoffs.

Centralized cloud computing sends collected information through a network to remote computing infrastructure. The remote servers perform the necessary processing and return results or make those results available to other applications. One advantage is that cloud infrastructure can provide substantial computing capacity and centralized management. It can also support systems where information from many locations needs to be collected and analyzed together.

AWS IoT Core is an example of technology that supports communication between connected devices and cloud services. It allows devices to exchange messages and integrate with additional cloud resources for processing or storage. This approach can be effective when a system needs greater computing capacity or centralized access to sensor information. However, its performance depends partly on the network connection. Even if a remote server processes information quickly, transferring data to the server and receiving a response requires additional time. Slow or unreliable network communication can therefore create delays.

Edge computing addresses some of these challenges by processing information closer to where it is collected. Instead of sending every sensor reading to a remote server, a nearby computer performs some or all of the required processing. This can reduce communication overhead and improve response times when local processing resources are sufficient. AWS IoT Greengrass demonstrates how applications can operate locally while still interacting with cloud services when needed.

However, edge computing also has limitations. Local computers may have less processing power, memory, and storage than remote servers. When a large number of sensor tasks arrive at the same time, the local computer may become overloaded. In this situation, tasks could spend more time waiting for processing than they would spend being transmitted to a remote server. As a result, processing information locally does not automatically guarantee better performance.

Another possible approach is hybrid computing, which combines local and remote processing. For example, tasks that require immediate responses could be processed locally, while tasks that require greater computing power could be sent to the cloud. Research involving iFogSim examines how computing resources can be distributed across different layers of a system, including edge, fog, and cloud environments. Fog computing generally refers to an intermediate computing layer between sensing devices and remote cloud infrastructure.

For my project, the main comparison will be between edge-only and cloud-only processing. Both approaches will be evaluated using the same simulated sensor workloads to compare task

completion times, network usage, and processing delays. A hybrid approach may be considered as a possible improvement later, but the initial investigation will focus on understanding the advantages and disadvantages of the two existing approaches.

What is the most important thing you still do not know?

The most important question I still need to investigate is under what conditions edge computing performs better than cloud computing, and when cloud processing becomes the more effective option.

Although existing research explains the advantages and limitations of both approaches, their performance depends on the system being evaluated. For example, a local computer may provide faster results when there are only a few sensors and the workload is relatively small. However, as the number of incoming tasks increases, its limited processing capacity may create delays. A cloud server may handle larger workloads more efficiently, but network communication could reduce that advantage.

To investigate this question, I plan to develop a simulation using Python and SimPy. SimPy is a discrete-event simulation library that allows developers to model processes, shared resources, and events occurring over simulated time. It is suitable for representing systems where multiple tasks compete for limited computing resources. This makes it useful for modeling sensors generating tasks and computers processing them.

The experiment will simulate a group of sensors producing tasks at different rates. Each task will require processing by either a local computer or a remote server. The local computer will have a lower processing capacity, while the remote server will have greater processing capacity but require additional network communication.

I plan to test several conditions, including low, medium, and high sensor workloads, along with different network delays. For example, an initial model could use an edge computer capable of processing five tasks per second and a cloud server capable of processing twenty tasks per second. Network delays could then be adjusted to observe how the results change. These values would be experimental assumptions rather than measurements from actual hardware.

The main measurements will include average task completion time, the percentage of tasks completed within a chosen time limit, the number of tasks waiting for processing, and the amount of information transmitted through the network. I will also examine unusually slow responses because average completion time alone may hide large delays affecting individual tasks.

To make the comparison fair, both processing approaches will receive the same generated sensor tasks, and each experiment will use identical workload conditions. I will repeat experiments

where randomized arrivals are used to check whether the results are consistent. The simulation will also use the same task sizes and processing requirements so that differences in performance can be connected to the computing resources and network conditions being tested.

The results will influence the design of the eventual system. If edge computing consistently produces faster responses at lower workloads but struggles when task arrival rates increase, this would suggest that processing decisions should consider available computing capacity. If cloud performance changes significantly with network delay, that would support including network conditions in any future task-placement strategy. However, the final design direction will depend on the evidence collected during the simulation.

Project Scope

My individual project will focus on building a simplified simulation that compares edge computing and centralized cloud computing for a network of sensors. The goal is to understand how processing capacity, sensor workload, and network communication affect system performance. This project will concentrate on a single type of simulated sensor-processing task to keep the experiments consistent and manageable.

The simulation will be developed using Python and the SimPy library. Sensors will be represented by software processes that generate tasks at predetermined or randomized intervals. Separate simulated computing resources will represent the local edge computer and remote cloud server. The model will include task queues, processing times, and network delays. Results will be collected and compared using tables and graphs.

The project will not require physical sensors, actual cloud servers, or a large distributed computing environment. It will also exclude detailed cybersecurity mechanisms, physical device energy measurements, and advanced machine-learning algorithms. These topics are important in real systems but would expand the project beyond its main research question.

A possible future improvement would be a basic scheduling method that decides whether to process a task locally or remotely based on the current workload and network conditions. However, this would only be considered after establishing the performance of the two existing approaches. The initial goal is not to create a completely new computing architecture but to understand the conditions under which each approach is effective.

This project is manageable for one student because the environment can be developed and tested entirely through software. Python and SimPy provide the tools needed to represent sensors, computing resources, and network delays without complicated hardware configurations. The limited number of variables also makes it possible to conduct repeatable experiments and explain the results clearly.

Overall, the project will provide a practical way to investigate performance tradeoffs in heterogeneous computing and sensing systems. By comparing existing approaches through simulation, I hope to develop a better understanding of how computing resources should be used when processing information from multiple sensors.

References

1. Shi, Weisong, et al. “Edge Computing: Vision and Challenges.” *IEEE Internet of Things Journal*, vol. 3, no. 5, 2016, pp. 637–646. https://doi.org/10.1109/JIOT.2016.2579198
2. Gupta, Harshit, et al. “iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in Internet of Things, Edge and Fog Computing Environments.” *Software: Practice and Experience*, vol. 47, no. 9, 2017, pp. 1275–1296. https://doi.org/10.1002/spe.2509
3. Amazon Web Services. “What Is AWS IoT Greengrass?” *AWS Documentation*. https://docs.aws.amazon.com/greengrass/v2/developerguide/what-is-iot-greengrass.html
4. Amazon Web Services. “What Is AWS IoT?” *AWS IoT Core Developer Guide*. https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html
5. SimPy Development Team. “SimPy Documentation: Overview.” *SimPy Documentation*. https://simpy.readthedocs.io/en/latest/